Big Data & BD NoSQL

Dr FERRAHI ibtisam
Maitre de conférences B, USTHB

2

MongoDB

JSON

- **JSON : JavaScript Object Notation.**
  - Format de données textuelles dérivé de la notation des objets du langage JavaScript.
  - Simple et complet pour la présentation des objets
  - Utilise des types de données connus et simples à décrire
  - Bonne lisibilité de la syntaxe
  - JSON est plus simple que XML, et il est très facile à associer à un langage de programmation.
  - JSON est massivement utilisé dans les applications web (AJAX), les web-services (REST), et … les bases de données NoSQL.

JSON

- Json se **base sur deux structures**
  - **Objet:** c'est une collection de couple Nom/Valeur
  - **Tableau:** c'est une liste de valeurs ordonnées
- **Exemple:**

objet

Couples nom/ valeur

```
{
  "ID_employé": 100,
  "Nom" : "ziani",
  "prenom":"yacine",
  "date_naiss": "2-05-1962",
  "tél": "15963"
} ,
{
  "ID_employé": 200,
  "Nom" : "Houloudi",
  "prenom":"Mohamed",
  "adresse": {
              "willaya":"Alger",
              "commune":"kouba"
              },
  "tél": "15963"
}
```


JSON

- **Json se base sur deux structures**
  - **Objet:** c'est une collection de couple Nom/Valeur
  - **Tableau:** c'est une liste de valeurs ordonnées
- **Exemple:**

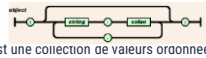
Couple nom/ valeur

```
{
  " Nom_série " : " La casa de papel ",
  "Acteur " : [" María Pedraza ","Miguel Herrán","Jaime Lorente"]
}
```

Tableau

JSON

- L'objet est un ensemble de couples Nom/ Valeur non ordonnée



- Un tableau est une collection de valeurs ordonnées :

- Une valeur peut être une ***chaîne de caractères***, ***un nombre***, ***true*** ou ***false*** ou ***null***, un ***objet***, un ***tableau*** :

Introduction

- **MongoDB (de Humongous, qui veut dire énorme)**
  - SGBD NoSQL orienté document à schéma flexible et distribuable
  - Ecrit en C++
  - Distribué sous licence AGPL
  - Développé en 2007 par firme MongoDB

SQL Vs. MongoDB

- **Table** = Collection
- **Enregistrement (tuple)** = Document
- **En BDR,** tous les tuples d'une table ***ont les mêmes champs*** (mais les valeurs peuvent être différentes (les valeurs sont affectées à des colonnes)
- **Dans une collection MongoDB,** les documents peuvent ne pas avoir un champ partage (pas de colonnes dans un document MongoDB).

Structure de données MongoDB

- Données représentées dans ***un schéma flexible***
- Données stockées sous forme de documents (clé valeur) JSON
- La structure de données n'est pas fixée à sa création
  - Mais en générale les documents d'une même collection ont une structure similaire (homogénéité)
- Les documents sont organisés sous forme de collections
  - Groupe de documents reliées par des indexes en commun

```
{   Nom : '' Ali'' ,
    Age : 25
    Section : '' A ''
    Modules : [  '' Bases de données '', '' Entrepôt de données '' ]
}
```

```
{   Nom : '' Mohamed'' ,
    Age : 22
    Section : '' A ''
    Modules : [  '' Bases de données '', '' Mathématique'' ]
}
```

Document MongoDB

- **Le champ _id**
  - Seul champ obligatoire, utilisé comme une clé primaire dans une collection
  - Peut être de tout type autre que tableau

- Les champs indexés ont une taille limite **(1 Mo)**
- Les noms des champs ne peuvent pas commencer par un **$**, contenir le caractère « , » ou le caractère **« null »**

Document MongoDB: Limites

- **Taille max d'un document: 16Mo**
  - Utilisation de l'api **Gridfs** pour stocker des documents plus larges que la taille autorisée
  - **Gridfs** permet de diviser les documents en **chunks** de même taille et les stocker sous forme de documents séparés

- **MongoDB préserve l'ordre des champs du document comme défini a leur création sauve:**
  - Que le champ **_id** doit toujours figurer en premier
  - Les opérations de update qui incluent le renommage d'un champ peut entrainer le changement de l'ordre des champs dans le document

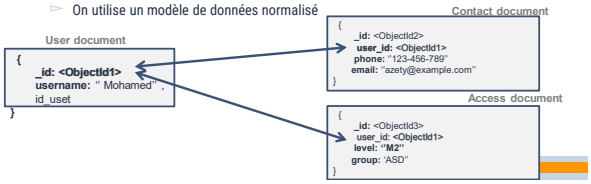
Document MongoDB: Limites

- **Champs _id**
  - Création d'un index unique à la création d'une collection
  - Utilisation conseillée d'un **Objectid:** objet petit ***(12o)*** unique et rapide à générer

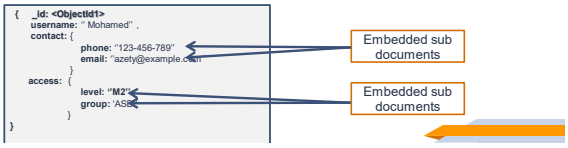
Structure de documents MongoDB

Deux façons des relations entre les données: **Références** et **données imbriquées**

- **Références**
  - Inclusion des liens ou références d'un document à un autre
  - C'est à l'application de résoudre ces références pour accéder aux données associées
  - On utilise un modèle de données normalisé



- **Données imbriquées (Embedded document)**
  - Sauvegarder les données associées dans la même structure de documents
  - Il est possible d'inclurent des documents dans un tableau ou un champ
  - Permet aux applications d'extraire et manipuler plusieurs niveaux de hiérarchies en une seule instruction
  - Ce sont des modèles de données dénormalisées



Comment choisir entre Références et données imbriquées?

- **On choisit les références quand:**
  - L'imbrication va produire des données dupliquées sans grand avantage en termes de performance de lecture
  - On veut représenter des relations **many-to-many** complexes
  - On veut représenter de larges ensembles de données hiérarchiques
- **On choisit les documents imbriqués quand:**
  - On a des relations « contains » entre les éléments (modèle one-to-one)
  - On a des relations **one-to many** ou les documents fils (many) apparaissent toujours dans le contexte des documents parents (one)

Opération de Lecture

- **Lecture**
  - Cible une unique collection spécifique de documents
  - Cible un critère ou des conditions spécifiques, pour identifier le document à retourner
  - Peut inclure une projection sur les champs du document à retourner
  - Peut définir des modificateurs pour imposer des limites, un ordre, un filtre

Collection **Query Criteria** **Modifier**

db.users.find ({ age: { **$gt** : 18 } }) . sort ({ age : 1 })

Opération de Lecture MongoDB

- MongoDB fournit une méthode db.collection.find() pour l'extraction de données
  - Accepte les critères de la requête, ainsi que les projections
  - Retourne un curseur vers les documents correspondants
- Fournit également une méthode db.collection.findOne() retournant un seul document

Opération de Lecture MongoDB

- **Projection**
  - Deuxième argument de la méthode find()
  - Peuvent spécifier la liste des champs à afficher ou à exclure
  - Mise à part pour exclure le champ _id (qui est affiché par défaut) il est interdit de mixer les projections inclusives et exclusives

Exemple:

Collections restriction projection

db . Users . find ({ age : 18 } , { name : 1, _id : 0 })

Opération d'Ecriture MongoDB

- **Modification**
  - Création, mise à jour ou suppression de données
  - Ces opérations modifient les données d'une seule collection
  - Définition de critère pour sélectionner les documents à modifier ou à supprimer
- **Exemple:**

```
db.users. insert (
    {
        name: ''brahim'',
        age: 26,
        stutus: ''Dr''
    }
)
```

Opération d'Ecriture MongoDB

- **Insertion de document**
  - Si vous voulez ajouter un document sans **champ _id**, le système l'ajoute lui-même en générant un champ de type **ObjectId**

```
db.users. insert (
    {
        name: ''brahim'',
        age: 26,
        stutus: ''Dr''
    }
)
```

Collection

Field: value
Field: value
Field: value

Document

Opération d'Ecriture MongoDB

- **Mise à jour de document**
  - Méthode **update** peut accepter des critères de sélection des documents à modifier, et des options qui affectent son comportement
  - Une opération update modifie un seul document par défaut (plusieurs docs si multi:true)
  - Existence de l'option ***upsert: true*** et que le document à modifier n'existe pas dans la même collection, il est automatiquement inséré

```
db.users. Update (
    {   age  :  {     $gt : 18}
},
    { $set: { status : ''Dr'' }     },
    { multi :true}
)
```

Collection
Update criteria
Update action
Update option

Opération d'Ecriture MongoDB

- **Suppression de documents**
  - La méthode **remove** supprime des documents d'une collection
  - Accepte des critères de sélection des documents à supprimer
  - Supprime par défaut tous les documents correspondant aux critères de sélection (sauf indication du contraire dans un flag approprié)
- **Exemple:**

```
db.users. remove (
   {
      stutus: "D"
   }
)
```

Collection

Remove criteria

Opération d'Ecriture MongoDB

- **Les opérations d'écriture sont atomiques au niveau d'un document**
  - Aucune opération de lecture ne peut affecter atomiquement plus qu'un seul document ou collection
  - L'utilisation des données imbriquées facilite l'écriture
  - La normalisation des données (référence vers données dans d'autres collections) implique plusieurs opérations d'écriture qui ne sont pas atomique
- **Certaine modification augmente la taille des documents**
  - Pour le moteur de stockage de MongoDB, si la taille d'un document dépasse la taille autorisée pour ce document, MongoDB le réalloue sur le disque
  - Si votre application nécessite plusieurs modifications qui augmente la taille des documents, considèrent plutôt l'utilisation des références

Mise a jours d'un enregistrement

- La mise à jours d'un enregistrement se fait par un UPDATE
- **Attention!** La mise a jour permet d'écraser tout l'enregistrement mais le _id restera le même

Suppression

- On peut supprimer :
  - Certains éléments d'une collection
  - Tous les éléments d'une collection
  - La collection entière (ses éléments et elle-même)

Suppression

- Supprimer un enregistrement donnée
  - **db.nom_collection.remove({nom : "Ali"}}**

Supprimer une BD

- Supprimer une bases de données **db.dropDatabase()**
  - Permet de supprimer la base de données en cours d'utilisation
  - MongoDB continue à la référencée → il faut passer à une autre base

Requête find()

- Commande "**db.collectionName.find({…},{…})**"
  - Le premier argument est la condition de **sélection/filtrage :**
    Seuls les enregistrements satisfaisant cette condition seront retenus
  - Le second argument (optionnel) décrit la projection : L'information voulue de chaque enregistremen t retenu après filtrage
- **Exemples :**
  - db.employes.find({date_recrutement : 2000}) :
  - db.employes.find({date : 2000},{nom : 1, _id : 0}) :

Opérateurs des requêtes

- **Liste des divers types d'opérateurs**
- Comparaisons :
  - **$eq, $ne** ==, !=
  - **$gt, $gte, $lt,$lte** >, >=, <, <=
  - **$in, $nin** Є, Є
- Logiques :
  - **$and, $or** AND, OR
  - **$not, $nor** NOT, NOR
- Op de test sur tableau :
  - **$all** Test sur le contenu
  - **$elemMatch** d'un tableau
  - **$size** Taille du tableau
- Op de test sur élément :
  - **$exists** Existence d'un champ ?
  - **$type** Teste le type d'un champ
- Op d'évaluation :
  - **$mod** Calcule et teste le résultat d'un modulo
  - **$regex** Recherche d'expression régulière
  - **$text** Analyse d'un texte
  - **$where**
    Test de sélectionne des enregistrements

Opérateur(s) AND

- **Cas 1:** L'opérateur AND est implicite à condition de porter sur les ***attribues différents***
  - Préciser plusieurs valeurs d'attributs dans le premier argument constitue un « AND »
  - **Db.employes.find**({**poste**: 'ing'', **date_rect** : 2011},{**nom** : 1, _id : 0})

Opérateur(s) AND

- **Cas 2: Réaliser plusieurs tests numériques sur un même attribut**
  - Préciser plusieurs valeurs d'attributs dans le premier argument constitue un « AND »
  - **db.employes.find**({poste : "ing", date_rect : {$gte : 2000, $lt : 2010}}, {nom : 1, _id : 0})

31

Opérateur(s) AND

- **Cas 3:** utiliser l'opérateur AND
  - Utiliser un opérateur $and explicite, mais il ne faut pas le mélanger avec une « , » :
  - **db.Employes.find**({**$and** : [{**poste** : "Ing"},
  - {**date_rect** : {$gte : 2000}},
  - {**date_rect** : {$lt : 2010}}]},
  - {**nom** : 1, _id : 0})
  - Tous les employés de la collection Employés dont le poste est ingénieur et la date de recrutement est comprise entre 2000 (inclus) et 2010 (exclus)

32

Opérateur OR

- Deux solutions pour exprimer un 'OR' :
  - **Avec l'opérateur "$in" :**
    - db.**Employés**.find({**date_rect** : {$in [2000, 2002, 2004]}},{**nom** : 1, _id : 0})
  - **Avec l'opérateur "$or" :**
    - db. **Employés**.find({**date_rect** : 2000,
      $or : [{**salaire** : {$lt : 100000}}, {**NbAbsences** : {**$lt** : 4}}]},
      {**nom** : 1, _id : 0})

33

Opérateur de test sur un élément

- **$exist** { fieldName : {**$exists** : boolean }}
  - {nbAbsences : {**$exists : true**, $lt 4}} → Si le champ nbAbsences existe, alors teste s'il est <4
  - {nbAbsence : {**$exists : false**}} → Si le champ nbEntrees n'existe pas alors retourne true
- **$type** { fieldName: {$type : BSON type number | String alias }}
  - {nbAbsences : {**$type : 16**} }
  - **Remarque :** type BSON numéro 16 (!) ↔ ''int''
  - {nbAbsences : {**$type : ''int''**}} → Si le champ nbAbsences est de type int, alors retourne true

34

- **$mod** { fieldName : { **$mod** : [divisor, remainder] }}
  - {nbEtudiants : { $mod : [3, 0] }}
  - Si le nombre d'étudiants est un multiple de 3, retourne true (exemple pour voir si on peut construire des trinômes)

35

- **$regex** est un opérateur de comparaison qui permet d'effectuer des recherches de motifs dans les champs de texte.
- **$regex** { fieldName : { $regex : regExp, $options: options }}
  - { regionName : { $regex : /^lor/ , $options : 'i' } }
    - **/^lor/ :** commence par ''lor''
    - $options : 'i' insensible à la casse
      - si le nom de la région commence par lor ou Lor ou LoR ou LOR alors retourne *true*
  - { regionName : { **$regex** : /lor/ , **$options** : 'i' } }
    - si le nom de la région contient lor ou Lor ou LoR ou LOR alors retourne *true*

36

- **$text** est utilisé pour effectuer des recherches textuelles dans une collection. Il permet de rechercher des documents qui contiennent un certain texte dans les champs indexés en texte.
  - Créer un index "text" sur le champs à analyser
    db.collection.createIndex("champsA" : "text")
  - Lancer un filtrage sur le contenu de ce champ (et sur d'autres)
  - **db.collection.find**({"champsB' : "valeurA",
    $text : {$search : "valeurB"})
  - 'champsB doit valoir "valeurA" ET " 'champsA" doit doit contenir 'valeurB"

37

- Exemple: soir la collection articles
- { "_id": 1, "title": "Introduction to MongoDB", "content": "MongoDB is a NoSQL database..." }
  { "_id": 2, "title": "Querying in MongoDB", "content": "Learn how to query data in MongoDB..." }
  { "_id": 3, "title": "Indexing in MongoDB", "content": "MongoDB supports various types of indexing..." }
- **db.articles.createIndex**({ content: "text" });
  - db.articles.find({ $text: { $search: "MongoDB" } });

38

- **Syntaxe générale de $text**
  - { **$text** : { **$search :** <string> ,
    $language : <string> ,
    $caseSensitive : <boolean>,
    $diacriticSensitive : <boolean>}}
- **$search**: C'est le champ dans lequel vous spécifiez la chaîne de caractères que vous recherchez.
- **$language**: [optionnel] permet de spécifier la langue pour la recherche textuelle.
- **$caseSensitive**: [booléen optionnel.]
  - **true**, la recherche prendra en compte la casse (distinguer majuscules et minuscules).
  - **false** (par défaut), la recherche est insensible à la casse.
- **$diacriticSensitive**: [booléen optionnel.]
  - **true**, la recherche prendra en compte les différences diacritiques (par exemple, la distinction entre "é" et "è").

39

- Exemple

```
{ "_id": 1, "title": "Introduction to MongoDB", "content": "MongoDB is a NoSQL database..." }
{ "_id": 2, "title": "Querying in MongoDB",     "content": "Learn how to query data in MongoDB..." }
{ "_id": 3, "title": "Indexing in MongoDB",     "content": "MongoDB supports various types of indexing..." }
```

```
db.collection.find({ $text: {
                        $search: "MongoDB",
                        $language: "english",
                        $caseSensitive: false,
                        $diacriticSensitive: false } });
```


Opérateurs d'évaluation

- Quand les opérateurs natifs de MongoDB ne suffisent plus, on peut introdire du code JavaScript
- **Attention:** c'est plus lent.
-

```
{ fieldName : {
                $where : JavaScript expression
              }
  }
}
```

  - **fieldName**: C'est le nom du champ sur lequel vous souhaitez appliquer la condition définie par l'expression JavaScript.
  - **$where**: C'est l'opérateur qui indique que vous allez utiliser une expression JavaScript pour effectuer la condition.
  - **JavaScript_expression**: C'est l'expression JavaScript qui sera évaluée pour chaque document. Elle doit retourner une valeur booléenne, **true** ou **false**, pour déterminer si le document satisfait la condition.

Opérateurs de test sur tableau

- **$all:** Un opérateur qui indique que tous les éléments spécifiés dans le tableau doivent être présents dans le champ tableau pour qu'un document soit retourné
- **Syntaxe :**

```
{ arrayFieldName : {
                     $all : [val1, val2, val3…]
                   }
}
```

  - Renverra true (retiendra l'enregistrement) si le champs arrayFieldName contient toutes les valeurs listées ensuite (val1, val2 et val3).

```
db.students.find({ cours: { $all: ["BDNoSQL", "SI"] } })
```


Opérateurs de test sur tableau

- **$elemMatch:** permet de spécifier plusieurs critères pour filtrer les éléments d'un tableau.
  - Syntaxe: { **arrayFieldName** : { **$elemMatch**: {query1, query2 …}}}
    - **arrayFieldName**: le nom du champs tableau dans votre collection a interroger
    - **{ query1, query2, ... }**: Ce sont les critères à appliquer à chaque élément du tableau.
    - Renverra ***true*** (retiendra l'enregistrement) si le champs **arrayFieldName** contient au moins un élément satisfaisant toute les requêtes.

- **Exemple 1:** on cherche s'il y a au moins une note dans l'intervalle [7,10[
  - Si le champs notes vaut [18, 8, 17, 11] → alors { notes : { $elemMatch : { **$gte** : 7, **$lt** : 10 } } } renverra ***true***

```
db.students.find({ grades: { $elemMatch:
                                { score: { $gt: 90 },
                                  subject: "Mathématiques "
                                }
                              }
})
```


Opérateurs de test sur tableau

- **$size** { arrayFieldName : { **$size** : theSize } }
  - Renverra ***true*** (retiendra l'enregistrement) si le champ **arrayFieldName** a la taille indiquée.

```
{ { "_id": 1,
    "name": "Aline",
   "grades": [ { "subject": "Mathématiques", "score": 92 },
               { "subject": "Physique", "score": 88 },
               { "subject": "Chimie", "score": 95 } ] },
 { "_id": 2,
   "name": "Nesrine",
   "grades": [ { "subject": "Mathématiques", "score": 88 },
               { "subject": "Physique", "score": 90 },
               { "subject": "Chimie", "score": 94 } ] },
{ "_id": 3,
  "name": "Ahmed",
  "grades": [ { "subject": "Mathématiques", "score": 94 },
              { "subject": "Chimie", "score": 89 }
            ]
} ]
```

```
db.students.find({ "grades": { $size: 2 } })
```

```
{"name": "Ahmed",
  "grades": [ { "subject": "Mathématiques", "score": 94 },
              { "subject": "Chimie", "score": 89 }
            ]
} ]
```


Opérateurs de test sur tableau

- **$size** { arrayFieldName : { **$size** : theSize } }
  - Renverra ***true*** (retiendra l'enregistrement) si le champ **arrayFieldName** a la taille indiquée.

```
db.students.find({ "grades": $size: { $gte: 2 }})
```

Application de méthodes aux collections

- **Syntaxe générale :**
  - **db.laCollection.find**(…).methode(…)
  - Ou bien: **db.laCollection.methode**(…)
  - La méthode peut être:
    - Sort()
    - Count()
    - forEach()

Application de méthodes aux collections

- **Méthode sort : db.collectionName.find(query, projection).sort(sortCriteria)**
  - Ordonne les enregistrements d'une collection selon un ou plusieurs champs, dans l'ordre croissant ou décroissant
    db.laCollection.find(…).sort({''ChampsA'' : 1, ''ChampsB'' : -1})
    - trie **laCollection** selon le champ **ChampsA** dans **l'ordre croissant**, et pour un même champs ChampsA, trie selon **champsB** dans **l'ordre décroissant**
- **Attention :** ~~db.laCollection.sort({''a.b.c.d'' : 1, ''a.b.x.y'' : -1})~~

Application de méthodes aux collections

- **Exemple sort : db.collectionName.find(query, projection).sort(sortCriteria)**
  - **db.laCollection.find(…).sort({''ChampsA'' : 1, ''ChampsB'' : -1})**
    - trie **laCollection** selon le champ **ChampsA** dans **l'ordre croissant**, et pour un même champs ChampsA, trie selon **champsB** dans **l'ordre décroissant**
  - **Exemple**

```
[ { "_id": 1, "name": "Aline", "ChampsA": 20, "ChampsB": 30 },
  { "_id": 2, "name": "Bahia", "ChampsA": 20, "ChampsB": 25 },
  { "_id": 3, "name": "Cerine", "ChampsA": 15, "ChampsB": 35 },
  { "_id": 4, "name": "Dahmane", "ChampsA": 15, "ChampsB": 20 }
]
```

```
db.students.find().sort({ 'ChampsA': 1, 'ChampsB': -1 })
```

```
[ { "_id": 4, "name": "David", "ChampsA": 15, "ChampsB": 20 },
  { "_id": 3, "name": "Charlie", "ChampsA": 15, "ChampsB": 35 },
  { "_id": 2, "name": "Bob", "ChampsA": 20, "ChampsB": 25 },
  { "_id": 1, "name": "Alice", "ChampsA": 20, "ChampsB": 30 } ]
```

Application de méthodes aux collections

- **Méthode count : db.laCollection.find(…).count()**
  - Compte le nombre d' enregistrements d'une collection
  - db.laCollection.find(…).**count()** Ou bien : db.laCollection.count()
  - Retourne le nombre d'enregistrements (de documents JSON) dans la collection
- **Exemple**
  - db.students.find().count()

52

Application de méthodes aux collections

- **Méthode forEach(*définition d'une fonction*)** :Définit et applique une fonction à chaque enregistre ment ou (document JSON) d'une collection
  - **Syntaxe : db.laCollection.find(…).forEach(function(doc) {…})**
  - **La fonction appliquée peut être sans action sur la collection**
    - **Exemple 1 :** db.laCollection.find().forEach(function(doc) {print(doc.a.b.c.d);})
    - **Exemple 2:**

      db.students.find({ score: { $gt: 80 } }).forEach(function(doc)

      { print("Student Name: " + doc.name + ", Score: " + doc.score); });
  - **Ou au contraire agir sur la collection (ex : la modifier).**
  - **Exemple 3: db.laCollection.find**().forEach(function(doc)

    { **db.laCollection.update** ({_id : doc._id}, { nouveau document : valeur }) })

53

- **Méthode forEach:** :Définit et applique une fonction à chaque enregistrement ou (document JSON) d'une collection
  - **Syntaxe : db.laCollection.find(…).forEach(function(doc) {…})**
  - La fonction appliquée peut être sans action sur la collection
  - **Exemple : db.laCollection.find**().**forEach(function(doc)**

    {print(doc.a.b.c.d);}
  - Ou au contraire agir sur la collection (ex : la modifier).
  - **Exemple : db.laCollection.find().**forEach(function(doc)

    { db.laCollection.update({_id : doc._id},

    { $set : {document : valeur} }) })
  - **Modifie un enregistrement (ne l'écrase pas complètement)**

54

Agrégations d'opérations avec aggregate (…)

Framework d'agrégation (1)

- **Principe :**
  - Le framework d'agrégation de MongoDB est un outil puissant qui permet de traiter et transformer des données de manière complexe.
  - Il fonctionne sur un système de pipeline, où les documents passent par plusieurs étapes, chacune effectuant une opération spécifique.
  - Le framework d'agrégation est flexible et expressif, adapté à un large éventail de tâches de traitement des données
- **Syntaxe: db.collectionName.aggregate**({op1}, {op2}, {op3}…)
- **La sortie d'une opération est l'entrée de la suivante**, ou bien la sortie finale de l'agrégation
- On peut pipeliner successivement autant d'opérations que l'on veut
- Plus lent que des commandes natives, car les données pipelinées ne profitent pas des index !
- Opérations pour l'agrégation :

  $match **$unwind** **$lookup**

Framework d'agrégation (1)

- **db.collectionName.aggregate**({op1}, {op2}, {op3}…)
- Opérations pour l'agrégation :

  $match **$unwind** **$lookup**
  $project **$sort**
  $group **$limit**

Framework d'agrégation (3)

- **$match :** Filtre les documents en fonction de critères spécifiés
- **Exemple 1:** On ne s'intéresse qu'aux employés dont la date de recrutement est >2011
  - **db.Employes.aggregate**([{"**$match**" : {"date_recrutement" : {$gt : "2011"}}}])
  - **db.getCollection('employes').aggregate**([{"**$match**" : {"date_recrutement" : {$gt : 2007}}}, { **$group:** { _id: null, **count**: { $sum: 1 } } }])

58

Framework d'agrégation (3)

- **$match :** Filtre les documents en fonction de critères spécifiés
- **Exemple 2:** On ne s'intéresse qu'aux employés dont la date de recrutement est >2011

```
db.ventes.aggregate([ { $match: { date: { $gte: ISODate('2022-01-01') } } },
                      { $group: { _id: "$produit", ventesTotales: { $sum: "$quantité" } } },
                      { $sort: { ventesTotales: -1 } }, { $limit: 5 } ]);
```

59

- **$project** :dans le framework d'agrégation de MongoDB est utilisée pour remodeler les documents dans le pipeline d'agrégation.
  - Cette phase vous permet d'inclure, exclure ou transformer des champs dans les documents résultants
  - Permet de projeter des attributs : seulement ceux voulus, et au besoin en les renommant ou en créant de nouveaux attributs
- **Exemple :** On ne retient que les employés dont l'année de recrutement est >2007
- **db.Employes.aggregate**({"$match" : {"date_recrutement" : {$gt : "2007"}}},
  {"$project" : {"nom" : 1, "date_recrutement" : 1}})

60

- **$group :**
  Permet de regrouper les enregistrements retenus selon l'_id (que l'on peut redéfinir au passage), et d'appliquer des fonctions de groupe au autres attributs projetés

```
db.Employes.aggregate( [
{"$match" : {" date_recrutement " : {$gt : "2007"}}},
{"$project" : {" fonction" : 1,  " date_recrutement  " : 1}},
 {"$group"   : {_id : "$fonction",
                "debut" : {$min : "$ date_recrutement"},
                 "occurences" : {$sum : 1}}
} ])
```

61

- **$sort :**
  Permet d'ordonner les enregistrements selon un des attributs, par ordre croissant (+1) ou décroissant (-1).

```
db.Employes.aggregate(
{"$match" : {" date_rectutement" : {$gt : "2007"}}},
 {"$project" : {" fonction" : 1,    " date_rectutement" : 1}},
  {"$group"   : {_id : "$fonction",
                 "debut" : {$min : "$date_recrutement"},
                  "occurences" : {$sum : 1}}},
 {"$sort"    : {"occurences" : -1}} )
```

62

- **$limit :** Permet de limiter le nombre de réponses aux plus importantes **Exemple :** on s'interesse qu'aux 4 fonctions les plus demandées

```
db.employes.aggregate( {"$match" : {"annee_recrutement" : {$gt : "2011"}}},
{"$project" : {"fonction" : 1, "annee_recrutement" : 1}}, {"$group"   : {_id : "$fonction",
                "debut" : {$min : "$annee_recrutement" "},
                 "occurences" : {$sum : 1}}}, {"$sort"    : {"occurences" : -1}}, {"$limit"    : 4} ).pretty()
```

63

- **$unwind :**
  Permet de remplacer un enregistrement contenant un tableau par une suite d'enregistrements contenant chacun un seul élément du tableau

- **$unwind**
  - Permet de remplacer un enregistrement contenant un tableau par une suite d'enregistrement contenant chacun un seul élément du tableau
  - Si on veut conserver les enregistrements des tableaux vides → on utilise une spécification de **$unwind**

Exemple : { "_id" : 1, "item" : "EFG", "sizes": [ ] }

db.theCollection.aggregate([

{ $unwind: { path: "$sizes", preserveNullAndEmptyArrays: true} }])

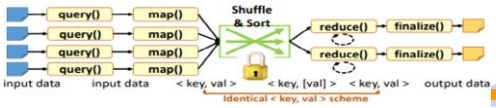
→ { "_id" : 2, "item" : "EFG" }

MapReduce de MongoDB

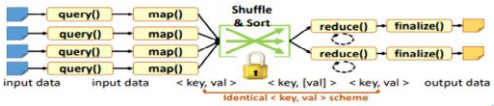
- **MongoDB possède son propre « Map-Reduce »** :
  - Ses propres règles de fonctionnement (un peu différentes d'Hadoop)
  - Ses propres règles de déploiement des tâches
  - Et son propre middleware sous-jacent
  - Fonctionne sur des bases distribuées (sharded)
  - N'exploite pas les index !

- Principes du mapReduce de MongoDB :
  - Une query pour pré-filtrer la collection traitée
  - Une fonction **map()**, en Java Script et qui accède à la base
  - Une fonction **reduce()**, en Java Script et qui ne doit PAS accéder à la base qui doit être commutative, associative et idempotente (!!)
  - Une fonction **finalize()**, en Java Script et qui ne doit pas accéder à la base
  - La possibilité de définir un ensemble de variables globales aux 3 fonctions map(), red uce() et finalize()

- **Map ():** fonctionne comme en Hadoop, mais sur une seule collection
  - Afin d'eviter d'implanter un filtrage en JS, une query en Mongo shell est applicable en amont
- **Reduce():** MongoDB applique reduce sur des <key, val> de sortie de map(), et sur des sorties précédentes de reduce():
  - Le format de sortie de reduce() doit être celui de sortie de map()
  - La fonction reduce() doit être commutative, associative et idempotente
- **Finalize():** permet de modifier la sortie finale de reduce sans contrainte



- Pas de combiner
- Pas d'optimisation possible du **shuffle & sort**
- **Les contraintes sur la fonction reduce()**
  - **Eviter les débordement mémoire:** en traitant chaque paire <clé, list de val > en plusieurs passes si la liste de valeurs est grande
  - **Limitent fortement les algorithmes possibles** en une seule opération map reduce



Exemple de mapReduce en MongoDB (compteur de livres)

- Deux syntaxes possibles d'exécution d'un MapReduce dans un mongo shell
  - Res=db.runCommand({"mapreduce":"Collection_name", "map":"map1", "reduce":"reduce1", "out":"res1", "finalize":"finalize1"});
  - db.collection_name.mapReduce(map1, reduce1, {"out":"res1", "finalize":"finalize1"});
- Code JavaScript entré dans un Mongo Shell

```
map1 = function() {
  var k = this.book.title;
  var v = {"count" : 1} ;
  emit(k,v);
}
```

```
reduce1 = function(key,values) {
  var s = 0;
  for (var i in values)
    s += values[i].count;
  return( {"count" : s} );
}
```

```
finalize1 = function (k,v) {
  return( {"title" : k, "nb" : v.count} );
}
```

70

Exemple de mapReduce en MongoDB (compteur de livres)

- Code JavaScript entré dans un mongo shell

```
Allure d'un Map :
function() {
  ...
  emit (this.a, val);
}
→ <key, val >
```

```
Allure d'un Reduce :
function(key, ListVals) {
  ...
  return ( val );
}
→ < key, val >
```

```
Allure d'un finalize :
function (k,v) {
  ...
  return ({....});
}
→ Sortie JSON : { "-id" : k ... , "value" : {....} }
```

- Deux syntaxes possibles d'exécution d'un mapReduce dans un mongo shell :
  - res= db.rumCommand ({"mapreduce" : "theCollection", "map" : map1, "reduce" : reduce1, "out" : "res1" , "finalize" : finalize1})
  - db.theCollection.mapReduce(map1, reduce1, {"out" : "res1" , "finalize" : finalize1})

71

Exemple de mapReduce en MongoDB :

SANS finalize() :

```
> b.res1.find()
{ "_id" : "titre1", "value" : { "count" : 10 } }
{ "_id" : "titre2", "value" : { "count" : 2 } }
....
```

AVEC finalize() : on peut modifier le document "value"

```
> b.res1.find()
{ "_id" : "titre1", "value" : {"title" : "titre1" , "nb" : 10 } }
{ "_id" : "titre2", "value" : {"title" : "titre2" , "nb" : 2 } }
....
```

La sortie d'un mapReduce est toujours un document JSON de format :

```
{ "_id" : xxxx, "value" : yyyy }
.....
```

72

Ajout d'une query sur les données d'entrée :

```
1  > res = db.runCommand(
            {"mapreduce" : "theCollection",
             "map" : map1,
             "reduce" : reduce1,
             "out" : "res1" ,
             "finalize" : finalize1},
             "query" : {"booktitle"  : "Cours de Big Data"}
     )

2  > db.theCollection.mapReduce(
            map1,
            reduce1,
            {"out" : "res1" , "finalize"  : finalize,
             "query" : {"booktitle"  : "Cours de Big Data"} }
     )
```